

RED TEAM LEARNING PATH

MODULE 05

Programming for Red Teamers

Fokus modul

- Menggunakan Bash, Python, dan PowerShell sebagai alat automation - bukan sekadar menghafal syntax.
- Membangun program kecil dengan input validation, predictable exit codes, structured output, logging, dan error handling.
- Memproses text, JSON, files, HTTP responses, process output, dan local socket state dengan cara yang aman.
- Menghindari command injection, unsafe shell invocation, accidental secret logging, dan dependency pollution.
- Membuat utility training yang dibatasi ke localhost / device sendiri sebagai fondasi untuk modul pentest berikutnya.

Repository: github.com/VaxXinE/red-teaming

Versi 1.0 - 04 Oktober 2026

Cara Menggunakan Modul Ini

Module 01-04 memberi lo terrain: Linux, network, HTTP/web, dan Windows. Module 05 mengubah pengetahuan itu menjadi automation. Tujuannya bukan menjadi software engineer penuh dalam satu modul. Tujuannya adalah bisa menulis utility kecil yang jelas kontraknya, aman terhadap input, mudah dites, dan menghasilkan evidence yang bisa dipercaya.

0.1 Learning Objectives

- [] Menentukan kapan Bash, Python, atau PowerShell menjadi pilihan paling tepat.
- [] Menulis Bash script dengan quoting, parameter validation, set -euo pipefail, exit code, dan cleanup yang masuk akal.
- [] Menggunakan Python standard library untuk argparse, pathlib, json, ipaddress, socket, urllib, hashlib, dan subprocess.
- [] Menjalankan external process dari Python tanpa shell=True untuk input yang tidak dipercaya.
- [] Menggunakan virtual environment atau uv agar dependencies tidak merusak system Python Arch.
- [] Menggunakan PowerShell objects, pipeline, ConvertTo-Json, try/catch, dan \$LASTEXITCODE secara benar.
- [] Mendesain output dengan stable schema dan timestamps agar bisa dipakai ulang.
- [] Menghindari secret leakage pada logs, command history, repo, dan evidence package.
- [] Membuat minimal satu cross-platform evidence workflow yang bisa direproduce.

LAB GATE

Semua network/socket/HTTP exercise di modul ini dibatasi ke localhost atau Windows VM milik lo sendiri. Utility yang disertakan sengaja menolak target non-loopback. Jangan menghapus guardrail tersebut untuk mengarahkannya ke sistem pihak lain tanpa authorization.

0.2 Tooling di Omarchy

Install / cek tooling utama

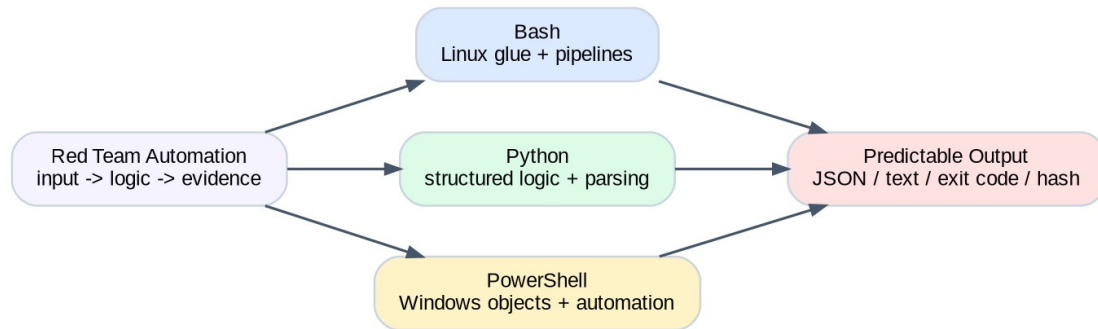
```
omarchy pkg add python uv shellcheck jq

python --version
uv --version
shellcheck --version
jq --version
```

Arch menyediakan Python, uv, pipx, dan ShellCheck di official repositories. Untuk project Python, kita prioritaskan isolated environment. Hindari `sudo pip install ...` ke system Python karena itu mengaburkan ownership package dan meningkatkan risiko dependency conflict.

Tool	Peran
Bash	Glue script, filesystem, process composition, pipelines, small automation di Omarchy.
ShellCheck	Static analysis untuk menemukan quoting/word-splitting dan bug shell umum.
Python	Structured logic, parsers, HTTP/socket client, automation yang lebih besar dan testable.
uv / venv	Isolated Python environment dan dependency management.
jq	Filter/transform JSON dari command line.
PowerShell	Object pipeline dan automation Windows VM.
Git	Versioning source code; secrets dan generated evidence jangan ikut commit.

1. Pilih Bahasa Berdasarkan Problem



Gambar 1 - Bash, Python, dan PowerShell punya sweet spot berbeda tetapi harus menghasilkan output yang predictable.

Situasi	Pilihan awal	Kenapa
Gabungkan command Linux + files	Bash	Sedikit logic, kuat untuk composition dan pipes.
Parsing JSON / banyak branching / reusable tool	Python	Data structures, library standard luas, testing lebih mudah.
Windows inventory / services / event logs	PowerShell	Cmdlets mengembalikan objects, bukan sekadar text.
Automation lintas Linux + Windows	Python + PowerShell helper	Pisahkan platform-specific collection dari normalization.

Rule of thumb

Kalau Bash mulai penuh nested quoting, parsing JSON dengan grep, atau array kompleks, pindahkan logic ke Python. Kalau Python cuma menjalankan tiga command berurutan tanpa data model, Bash mungkin lebih sederhana.

2. Kontrak Program: Input, Output, Exit Code

Script yang berguna punya kontrak. Input harus jelas, output harus bisa diprediksi, error tidak boleh diam-diam dianggap sukses, dan exit status harus mewakili hasil eksekusi. Ini penting saat script nanti dirangkai dengan CI, shell pipeline, atau tooling lain.

```

# Success
exit 0

# Usage / bad arguments (contoh konvensi sysexits)
exit 64

# Generic operational failure
exit 1

```

Security perspective

Ambiguous failure adalah risiko security. Kalau collection gagal tetapi script tetap menulis report seolah lengkap, analyst dapat membuat keputusan berdasarkan evidence palsu atau parsial.

3. Bash Fundamentals untuk Automation

3.1 Shebang, strict-ish mode, dan quoting

```
#!/usr/bin/env bash
set -euo pipefail
umask 077

name=${1:-}
[[ -n "$name" ]] || { printf 'name required\n' >&2; exit 64; }
printf 'hello %s\n' "$name"
```

`set -e` menghentikan banyak kasus command gagal, `-u` memperlakukan variable unset sebagai error, dan `pipefail` membuat pipeline gagal bila komponen kanan-kiri yang relevan gagal. Tetapi `-e` punya exception rules; jangan memperlakukannya sebagai pengganti explicit error handling.

Common mistake

Unquoted expansion seperti `rm -rf \$dir/\*` berbahaya karena word splitting dan globbing dapat mengubah arti input. Biasakan `"\$var"`, prefix `--` sebelum path argument, dan guard khusus untuk operasi destruktif.

3.2 Functions, arrays, dan temporary files

```
cleanup() {
  [[ -n "${tmpdir:-}" ]] && -d "$tmpdir" ]] && rm -rf -- "$tmpdir"
}
trap cleanup EXIT

tmpdir=$(mktemp -d)
args=(--numeric --brief)
printf 'temporary workspace: %s\n' "$tmpdir"
```

Array Bash mencegah lo membangun satu command string besar. Untuk operasi sensitif, pikirkan lifecycle temporary data: dibuat di mana, permission-nya apa, kapan dihapus, dan apakah bisa berisi credential material.

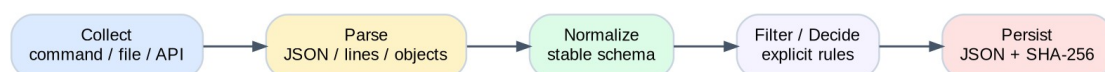
4. Text Pipelines tanpa Membuat Parser Rapuh

```
printf '%s\n' alpha beta gamma | grep 'a' | sort

ss -ltn | awk 'NR == 1 || /127\.0\.0\.1/'

printf '{"status":"ok","count":3}\n' | jq -r '.status'
```

`grep`, `awk`, `sed`, `sort`, `cut`, dan `jq` adalah building blocks. Tetapi parser text harus mengikuti format yang stabil. Jika command menyediakan JSON/XML/structured output, prioritaskan format structured daripada regex terhadap tampilan manusia.



Gambar 2 - Automation yang maintainable memisahkan collect, parse, normalize, decision, dan persistence.

5. Python Environment di Arch / Omarchy

Gunakan isolated environment untuk project. Python `venv` membuat interpreter/site-packages terpisah. `uv` menyediakan workflow cepat untuk membuat environment dan mengelola dependencies. Untuk CLI pihak ketiga yang ingin lo install sebagai aplikasi, `pipx` juga cocok karena setiap app memiliki environment terisolasi.

Environment minimal dengan uv

```
mkdir -p ~/CyberSec/scripts/module05/demo
cd ~/CyberSec/scripts/module05/demo

uv venv .venv
source .venv/bin/activate
python --version

deactivate
```

Supply-chain habit

Dependency adalah executable code. Pin/lock versi bila project mulai serius, review package provenance, dan jangan copy-paste installer `curl | sh` tanpa memahami sumber serta efeknya.

6. Python: argparse, pathlib, JSON, dan Data Model

```
from pathlib import Path
import argparse
import json

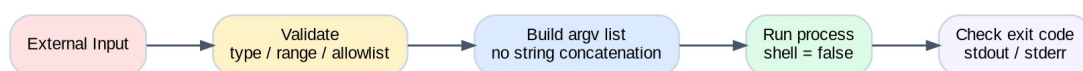
parser = argparse.ArgumentParser()
parser.add_argument('path', type=Path)
args = parser.parse_args()

if not args.path.is_file():
    parser.error('path must be a file')

payload = {
    'name': args.path.name,
    'size': args.path.stat().st_size,
}
print(json.dumps(payload, indent=2))
```

`argparse` memberi usage/help dan validation surface. `pathlib` mengurangi string path manipulation. JSON cocok sebagai stable machine-readable output selama schema-nya jelas dan tidak berubah sembarangan.

7. Secure Subprocess: Jangan Jadikan Input sebagai Shell



Gambar 3 - External input harus divalidasi dan dipetakan menjadi argv list; shell tidak perlu terlibat untuk sebagian besar automation.

```
completed = subprocess.run(
    ['ip', 'route', 'show'],
    capture_output=True,
    text=True,
    timeout=5,
    check=False,
)

if completed.returncode != 0:
    raise RuntimeError(completed.stderr)
```

Python tidak memilih system shell secara implicit ketika lo memberikan argument list. Risiko meningkat ketika code memanggil `shell=True` lalu memasukkan string yang berasal dari user/external source. Metacharacters shell dapat berubah menjadi syntax, bukan data.

Anti-pattern

Jangan membangun `command = "tool " + user\_input` lalu `subprocess.run(command, shell=True)`. Lebih aman validasi input dan kirim setiap argument sebagai item list.

8. Socket Fundamentals untuk Local Lab

Socket programming membantu menghilangkan kesan bahwa Nmap atau client HTTP adalah magic. Kita cukup memahami connect -> timeout -> result. Utility repo sengaja menolak non-loopback address supaya exercise tetap lokal.

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
    sock.settimeout(0.4)
    result = sock.connect_ex(('127.0.0.1', 8000))

print('open' if result == 0 else 'closed')
```

Boundary

Code di atas adalah konektivitas TCP sederhana, bukan scanner produksi. Di modul enumeration nanti kita akan belajar metodologi dan tooling yang tepat pada authorized lab.

9. HTTP Client dengan Python Standard Library

```
from urllib.request import Request, urlopen

req = Request(
    'http://127.0.0.1:8000/',
    headers={'User-Agent': 'module05-lab/1.0'},
)
with urlopen(req, timeout=3) as response:
    print(response.status)
    print(response.headers.get('Content-Type'))
```

Untuk utility internal kecil, standard library sering cukup. Kalau nanti memakai library pihak ketiga seperti `httpx` atau `requests`, gunakan isolated environment dan lock dependency.

10. Logging, Evidence, Hashing, dan Secret Hygiene

Logs seharusnya membantu reproduce sebuah run: timestamp, action, target yang memang authorized, exit status, dan output penting. Tetapi logs tidak boleh menjadi tempat dump token, cookies, API keys, Authorization headers, password, atau private key.

```
from hashlib import sha256
from pathlib import Path

path = Path('evidence.json')
digest = sha256(path.read_bytes()).hexdigest()
print(f'{digest} {path}')
```

Yang dicatat	Yang sebaiknya di-redact / tidak dicatat
Timestamp UTC, tool version, input non-secret	Password, bearer token, session cookie
Exit code, error category, file hash	Private key, recovery code, secret environment variables
Authorized target identifier	Full credential dumps atau browser secrets

11. PowerShell sebagai Object Programming Environment

PowerShell berbeda dari text shell tradisional karena cmdlet mengirim objects dengan properties dan methods melalui pipeline. Ini membuat filtering dan serialization lebih reliable dibanding parsing kolom text.

```
Get-Process |
Sort-Object CPU -Descending |
Select-Object -First 10 Name, Id, CPU |
ConvertTo-Json -Depth 3
```

Gunakan `Get-Member` saat lo belum tahu properties sebuah object. Untuk error yang ingin ditangkap, banyak cmdlet perlu `-ErrorAction Stop` agar non-terminating error dipromosikan menjadi terminating error.

```
try {
    Get-Item 'C:\NoSuchFile.txt' -ErrorAction Stop | Out-Null
}
catch {
    Write-Warning "Read failed: $($_.Exception.Message)"
}
finally {
    Write-Verbose 'cleanup complete' -Verbose
}
```

Native command nuance

Program native melaporkan failure lewat exit code. PowerShell menyimpannya di `\$LASTEXITCODE`; jangan berasumsi `try/catch` otomatis menangkap semua non-zero native exit code.

12. Functions yang Mudah Diuji

Pisahkan logic murni dari I/O. Function yang menerima value dan mengembalikan value lebih mudah dites daripada function yang sekaligus membaca keyboard, memodifikasi filesystem, memanggil network, dan print output.

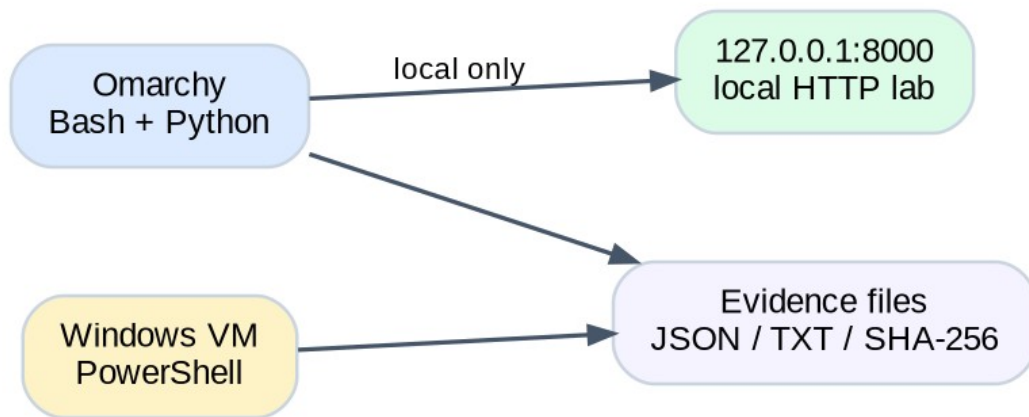
```
def normalize_port(value: int) -> int:
    if not 1 <= value <= 65535:
        raise ValueError('invalid TCP/UDP port')
    return value

def test_normalize_port():
    assert normalize_port(443) == 443
```

Engineering mindset

Red Team utility tetap software. Small tests, type hints, linting, dan deterministic output mengurangi human error saat engagement sebenarnya.

13. Cross-platform Automation Pattern



Gambar 4 - Module 05 memakai Omarchy, localhost HTTP lab, dan Windows VM; semua output dinormalisasi menjadi evidence files.

Pattern yang akan kita bawa ke modul berikutnya: platform-specific collector menghasilkan structured output; normalization dilakukan terpisah; evidence diberi timestamp/hash; lalu analyst mengambil keputusan. Hindari satu monster script yang melakukan semuanya dengan side effects tersembunyi.

14. Hands-on Lab 1 - Tooling & Workspace

Objective

Siapkan workspace programming yang tidak mencampur dependency project dengan system Python.

```
mkdir -p ~/CyberSec/scripts/module05/{bash,python,evidence}
cd ~/CyberSec/scripts/module05

omarchy pkg add python uv shellcheck jq
python --version
uv --version
```

- [] Workspace ada.
- [] Tool versions dicatat di notes.
- [] Tidak ada sudo pip install.
- [] Git repo tidak berisi .env / evidence sensitif.

15. Hands-on Lab 2 - Bash Input Validation

```
cp /path/to/repo/labs/evidence-pack.sh ~/CyberSec/scripts/module05/bash/
cd ~/CyberSec/scripts/module05/bash
shellcheck evidence-pack.sh
./evidence-pack.sh
```

Tanpa dua arguments, script harus menolak usage dengan exit code non-zero. Setelah itu buat folder dummy dan paketkan:

```
mkdir -p ~/CyberSec/scripts/module05/evidence/sample
printf 'hello\n' > ~/CyberSec/scripts/module05/evidence/sample/note.txt
./evidence-pack.sh \
~/CyberSec/scripts/module05/evidence/sample \
~/CyberSec/scripts/module05/evidence/packages
```

Verifikasi manifest dengan `sha256sum -c <manifest-file>`. Perhatikan bahwa script menggunakan quoting, `--`, `realpath`, `umask 077`, dan menolak `)` sebagai input.

16. Hands-on Lab 3 - Bash Pipeline & jq

```
printf '%s\n' \
'{"name":"web","port":8080,"state":"open"}' \
'{"name":"ssh","port":22,"state":"closed"}' \
'{"name":"api","port":3000,"state":"open"}' \
> services.jsonl

jq -s '[] | select(.state == "open")]' services.jsonl
```

Simpan output sebagai `open-services.json`. Lalu hash file tersebut. Goal-nya adalah melihat bedanya parsing structured data dibanding grep terhadap tampilan manusia.

17. Hands-on Lab 4 - Python Environment + argparse

```
cd ~/CyberSec/scripts/module05/python
uv venv .venv
source .venv/bin/activate
python -m pip --version || true
python - <<'PY'
import sys
print(sys.executable)
PY
```

Kalau environment dibuat oleh uv dan pip tidak terpasang, itu bukan masalah untuk lab standard-library ini. Fokus kita adalah isolation dan interpreter path.

18. Hands-on Lab 5 - Safe Subprocess

```
cp /path/to/repo/labs/safe-process.py .
python safe-process.py --help
python safe-process.py identity
python safe-process.py routes
```

Buka source dan jawab: di mana allowlist command berada? Kenapa user tidak bisa memasukkan arbitrary command? Kenapa `shell=True` tidak diperlukan? Ubah hanya output formatting, jangan hilangkan allowlist.

19. Hands-on Lab 6 - Local TCP Checker

Terminal A: buat service loopback:

```
python -m http.server 8000 --bind 127.0.0.1
```

Terminal B: jalankan utility:

```
python local-port-check.py --port 8000 --port 9000
python local-port-check.py --host localhost --port 8000
```

Sekarang coba target non-loopback, misalnya alamat IP LAN lo sendiri. Utility harus menolak sebelum membuat koneksi. Itu intentional guardrail, bukan bug.

20. Hands-on Lab 7 - Local HTTP Inspector

```
python http-inspect.py http://127.0.0.1:8000/
python http-inspect.py http://localhost:8000/
```

Output harus berupa JSON ringkas berisi status, Content-Type, Server, dan sample byte count. Script membatasi scheme ke HTTP dan host ke loopback.

Secure design lesson

SSRF prevention dimulai dari boundary yang eksplisit. Untuk training utility ini, allowlist paling sederhana adalah loopback saja. Di production, validation harus mempertimbangkan DNS rebinding, redirects, address families, proxies, dan network policy.

21. Hands-on Lab 8 - Hash & Evidence Integrity

```
python local-port-check.py --port 8000 > evidence.json
sha256sum evidence.json > evidence.json.sha256
sha256sum -c evidence.json.sha256
```

Edit satu karakter pada copy file lalu jalankan check lagi. Hash mismatch membuktikan file berubah, bukan membuktikan siapa yang mengubah atau apakah content awal benar. Integrity hash punya scope yang spesifik.

22. Hands-on Lab 9 - PowerShell JSON Collector

Di Windows VM Module 04, copy `windows-inventory-json.ps1` ke Documents lalu jalankan:

```
Set-ExecutionPolicy -Scope Process Bypass
cd "$env:USERPROFILE\Documents\RedTeam\Module05"
.\windows-inventory-json.ps1
Get-Content .\windows-inventory.json -Raw | ConvertFrom-Json | Format-List
```

Perhatikan bahwa PowerShell mengumpulkan objects lalu serialize ke JSON. Hash ditampilkan setelah file ditulis. Script tidak membaca credentials atau secret stores.

23. Hands-on Lab 10 - PowerShell Error Handling

```
try {
    Get-Item 'C:\Definitely-Missing-Module05.txt' -ErrorAction Stop
}
catch {
    [pscustomobject]@{
        ok = $false
        type = $_.Exception.GetType().FullName
        message = $_.Exception.Message
    } | ConvertTo-Json
}
```

Ulangi tanpa `-ErrorAction Stop` dan amati bedanya. Ini membantu memahami non-terminating vs terminating error sebelum lo menulis automation Windows yang besar.

24. Mini Project - Evidence Workflow

Scenario

Buat evidence baseline sederhana dari Omarchy dan Windows VM, tetapi hasil akhirnya harus machine-readable, hashed, dan tidak berisi secret.

1. Di Omarchy, jalankan `local-port-check.py` dan simpan JSON.
2. Tambahkan satu output read-only dari `safe-process.py`.
3. Buat folder evidence dengan nama timestamp UTC.
4. Hash seluruh file evidence dengan `sha256sum`.
5. Gunakan `evidence-pack.sh` untuk membuat archive + manifest.
6. Di Windows VM, jalankan `windows-inventory-json.ps1`.
7. Copy JSON Windows melalui shared folder yang memang disediakan lab, bukan lewat random public upload service.
8. Tuliskan README singkat: waktu capture, host, tools, exit status, dan apa yang sengaja tidak dikumpulkan.

Do not collect

Jangan memasukkan ``.env``, browser profile, cookies, password store, private keys, token, recovery keys, atau credential dump ke evidence package latihan.

25. Practical Challenge - Build a Local Inspector

Buat program Python baru bernama `local-inspector.py` dengan requirement berikut:

- [] Menerima `--port` lebih dari sekali dan validasi 1..65535.
- [] Target hard default 127.0.0.1; jika menerima `--host`, hanya loopback yang boleh.
- [] Mencatat generated_at UTC.
- [] Mencetak JSON stable schema ke stdout.
- [] Jika input invalid, keluar non-zero dengan pesan ke stderr.
- [] Tidak memakai `shell=True`.
- [] Timeout dapat diatur tetapi dibatasi maksimum 5 detik.

[] Memiliki minimal tiga test cases manual: open, closed, invalid target.

Stretch goal

Tambahkan `--output FILE` dengan permission restrictive dan SHA-256. Jangan tambahkan CIDR sweep atau remote scanning; itu materi enumeration terpisah pada authorized lab.

26. Code Review Checklist untuk Utility Red Team

Area	Checklist
Input	Type/range validated? allowlist possible? path traversal considered?
Process exec	argv list? shell disabled? timeout? exit code checked?
Filesystem	Safe path? restrictive permission? atomic-ish write? cleanup?
Network	Authorized target? timeout? redirects? loopback/lab guardrail where appropriate?
Secrets	No hard-coded keys? no token logging? no secrets in Git?
Output	Stable schema? UTC timestamp? explicit error path?
Dependencies	Official source? isolated environment? version documented?
Evidence	Hash recorded? provenance noted? partial failures visible?

27. Knowledge Check

1. Kapan Bash lebih cocok daripada Python?
2. Apa konsekuensi word splitting pada variable Bash yang tidak di-quote?
3. Apa fungsi pipefail?
4. Kenapa set -e bukan pengganti error handling?
5. Apa keuntungan structured output dibanding parsing tampilan manusia?
6. Kenapa system Python Arch sebaiknya tidak dipenuhi package pip global?
7. Apa fungsi argparse?
8. Kenapa subprocess argument list lebih aman daripada shell=True + string?
9. Apa beda stdout dan stderr?
10. Kenapa timeout penting pada network/process automation?
11. Apa manfaat ipaddress untuk validation?
12. Kenapa hash bukan bukti authenticity?
13. Apa yang bergerak di PowerShell pipeline: text atau objects?
14. Kenapa -ErrorAction Stop sering dipakai di dalam try?
15. Apa fungsi \$LASTEXITCODE?
16. Kenapa log tidak boleh dump seluruh environment variables?
17. Apa manfaat stable JSON schema?
18. Kenapa small pure function lebih mudah dites?
19. Bagaimana cara mengurangi risiko supply-chain dependency?
20. Apa tiga guardrail yang harus dimiliki utility training yang menyentuh network?

28. Exit Criteria

- [] Bisa menulis Bash script kecil yang lolos ShellCheck tanpa bug quoting utama.
- [] Bisa menjelaskan kapan pindah dari Bash ke Python.
- [] Bisa membuat isolated Python environment dengan uv/venv.

- [] Bisa menulis argparse + validation + JSON output.
- [] Bisa menjalankan subprocess dengan argv list dan memeriksa return code.
- [] Bisa membuat TCP connection sederhana ke localhost dan memahami timeout.
- [] Bisa membuat HTTP request lokal dan memvalidasi target boundary.
- [] Bisa serialize PowerShell objects ke JSON dan menangani terminating errors.
- [] Bisa menghasilkan evidence + SHA-256 tanpa mengumpulkan secrets.
- [] Bisa melakukan code review dasar menggunakan checklist security di modul ini.

29. Cheat Sheet

```
# Bash
set -euo pipefail
umask 077
shellcheck script.sh
mktemp -d
trap cleanup EXIT

# Python env
uv venv .venv
source .venv/bin/activate
python script.py --help

# JSON
jq . file.json
python -m json.tool file.json

# Hash
sha256sum evidence.json
sha256sum -c evidence.json.sha256

# Local lab
python -m http.server 8000 --bind 127.0.0.1
python local-port-check.py --port 8000
python http-inspect.py http://127.0.0.1:8000/

# PowerShell
Get-Command
Get-Help <cmdlet> -Full
<object> | Get-Member
$data | ConvertTo-Json -Depth 5
Get-FileHash -Algorithm SHA256 <file>
```

30. Referensi Utama

GNU Bash Reference Manual - The Set Builtin - Semantics set -e, -u, pipefail, dan shell options.

https://www.gnu.org/software/bash/manual/html_node/The-Set-Builtin.html

Python documentation - subprocess - API dan security considerations untuk process execution.

<https://docs.python.org/3/library/subprocess.html>

Python documentation - venv - Virtual environments untuk isolated project dependencies.

<https://docs.python.org/3/library/venv.html>

Python documentation - argparse - Command-line parsing dan validation.

<https://docs.python.org/3/library/argparse.html>

Python documentation - ipaddress - IP address/network objects dan validation.

<https://docs.python.org/3/library/ipaddress.html>

Microsoft Learn - about_Objects - Object model PowerShell.

https://learn.microsoft.com/powershell/module/microsoft.powershell.core/about/about_objects

Microsoft Learn - about_Pipelines - Cara objects bergerak melalui pipeline.

https://learn.microsoft.com/powershell/module/microsoft.powershell.core/about/about_pipelines

Microsoft Learn - about_Error_Handling - Terminating/non-terminating/native command errors.

https://learn.microsoft.com/powershell/module/microsoft.powershell.core/about/about_error_handling

Arch Linux package - Python - Python package resmi Arch.

https://archlinux.org/packages/core/x86_64/python/

Arch Linux package - uv - uv package resmi Arch.

https://archlinux.org/packages/extra/x86_64/uv/

Arch Linux package - ShellCheck - Shell static analysis package resmi Arch.

https://archlinux.org/packages/extra/x86_64/shellcheck/

Omarchy Manual - Other Packages - Package workflow dan AUR caution.

<https://omarchy.org/manual/other-packages/>

Next module

Module 06: Pentest Methodology, Scope & Rules of Engagement. Setelah bisa automate dengan disiplin, kita masuk lifecycle engagement: authorization, scoping, assumptions, evidence handling, testing windows, communication, cleanup, dan reporting expectations.